

## Fall 2025 (A)

1. a) Suppose you are running a background data processing program that spawns a child process to handle file uploads. Suddenly, the parent process crashes before finishing and never calls `wait()` to check the child's exit status. Write what happens to the child process in this situation?

-If the parent process crashes, the child becomes an orphan process. It continues executing and is adopted by the system process (`init/systemd`). After completion, it is automatically cleaned up by the system.

b) Write the outputs in the question paper. The output CA-M [1\*3  
sequence should be exactly matched. =3]

Write the outputs of each print statement in one line.

```
main:
    int p = 2, q = 3, r = 4
    pid1 = fork()
    if (pid1 == 0):
        p = p + 3
        q = q + 1
        r = r + 2
        print("p:", p, " q:", q, " r:", r)
    else:
        wait(NULL)
        p = p + 4
        q = q + 5
        r = r + 6
        pid2 = fork()
        if (pid2 == 0):
            p = p + 1
            q = q + 1
            r = r + 1
            print("p:", p, " q:", q, " r:", r)
        else:
            wait(NULL)
            print("p:", p, " q:", q, " r:", r)
```

Ans:

p:5, q:4, r:6

p:7, q:9, r:11

p:6, q:8, r:10

TOPIC NAME: Fall 2025 (A)

DAY: Azmari Sultana

TIME: \_\_\_\_\_

DATE: / /

---

① b)

```
1 main:
2   int p=2, q=3, r=4
3   pid1 = fork()
4   if (pid1 == 0):
5       p = p+3
6       q = q+1
7       r = r+2
8       printf("p: ", p, "q: ", q, "r: ", r)
9   else:
10      wait(NULL)
11      p = p+4
12      q = q+5
13      r = r+6
14      pid2 = fork()
15      if (pid2 == 0):
16          p = p+1
17          q = q+1
18          r = r+1
19          printf("p: ", p, "q: ", q, "r: ", r)
20      else:
21          wait(NULL)
22          printf("p: ", p, "q: ", q, "r: ", r)
```

Output:

p: 5, q: 4, r: 6 → P2  
p: 7, q: 9, r: 11 → P3  
p: 6, q: 8, r: 10 → P1

P1  
p=26  
q=28  
r=10  
pid1=P2  
pid2=P3

P2  
pid1=0  
p=25  
q=24  
r=6

P3  
p=67  
q=59  
r=11  
pid2=0

GOOD LUCK™

**c) Adhara wants to build a media editing application where multiple processes such as audio, video and effects processes work on the same data simultaneously. Since the data needs to get accessed and updated frequently, she needs a communication method that ensures high speed with minimal kernel support.**

**I. Identify the suitable interprocess communication method for Adhara in this scenario.**

**II. State the reason behind the selection.**

I. The suitable IPC method is Shared Memory.

II. Because shared memory allows multiple processes to access and modify the same data directly from memory, it provides very high speed communication with minimal kernel involvement (no repeated data copying).

**2.a) Dipu has implemented two identical MLFQ schedulers. In the first implementation, new processes arrive at the queue having the highest priority level. In the second implementation, new processes arrive at the queue having the lowest priority level. Apart from the mentioned difference, all other operations of MLFQ are the same as regular MLFQs. A balanced number of I/O and CPU-bound processes arrive in the same order in both implementations of Dipu. Explain how the different scheduling criteria would differ across both implementations.**

♦ **First Implementation (Highest Priority Start)**

New processes immediately get CPU, resulting in fast response time. I/O-bound processes benefit and remain in higher queues, while CPU-bound processes are demoted quickly. This may cause starvation of long CPU-bound processes and increases context switching.

♦ **Second Implementation (Lowest Priority Start)**

New processes must wait, leading to slower response time. CPU-bound processes get a fairer share of CPU, reducing starvation. I/O-bound processes may be delayed, but overall scheduling becomes more balanced and fair.

**b) Write the answers in the answer script.**

**CA-E**

In an OS, **Multilevel Feedback Queue** functions as a scheduler. In the scheduler, the ready queue is divided into **three** levels. At every level, the round-robin algorithm has been employed, and certain constraints must be maintained, as listed below.

- **Q0:** time quantum = **4**, priority = **0**
- **Q1:** time quantum = **6**, priority = **1**
- **Q2:** time quantum = **8**, priority = **2**

Here, the lower priority value indicates a higher priority, and in the case of an I/O request, the scheduler will promote a process by one level upon completion of the I/O operation. The information on burst times, arrival times, and I/O times of three processes is given below:

| Process | Burst Time | Arrival Time | I/O Time                                      |
|---------|------------|--------------|-----------------------------------------------|
| P1      | 10         | 4            | 3s I/O [after total <b>8s</b> CPU allocation] |
| P2      | 11         | 0            | N/A                                           |
| P3      | 11         | 6            | 5s I/O [after total <b>9s</b> CPU allocation] |

- Draw** a Gantt chart using the multilevel feedback queue scheduling algorithm, showing the states of the ready queues of different levels.
- Calculate** the average waiting time and average turnaround time from the Gantt chart.

**[5]**

**[1+1  
=2]**

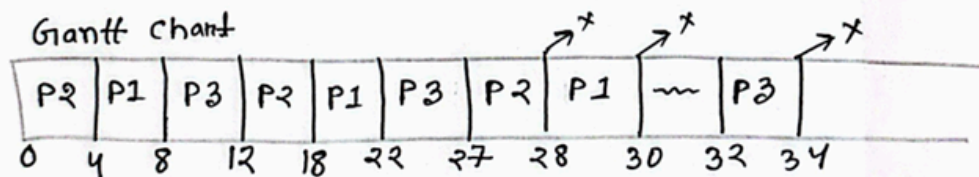
# Fall 2025 (A) → CPU Scheduling

Q b) (i)

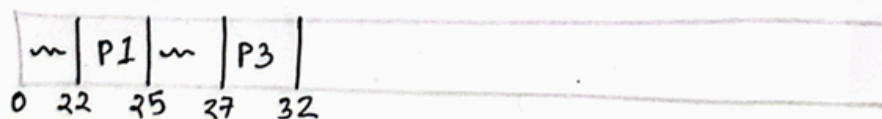
| Process | Arrival time | Burst time                 | I/O time                    |
|---------|--------------|----------------------------|-----------------------------|
| P1      | 4            | <del>10</del> 8 0<br>8 4 0 | 3s I/O (after total 8s CPU) |
| P2      | 0            | 4 7 0                      | N/A                         |
| P3      | 6            | 4 7 0<br>9 5 0             | 5s I/O (after total 9s CPU) |

| Q0 (RR:4)  | Q1 (RR:6)      | Q2 (RR:8) | I/O     |
|------------|----------------|-----------|---------|
| 0: P2      | 4: P2          | 18: P2    | 23: P1  |
| 4: P1      | 8: P2, P1      | 22: P2    | 25: { } |
| 6: P3      | 12: P2, P1, P3 | 27: P2    | 27: P3  |
| 12: { }    | 18: P1, P3     | 28: { }   | 32: { } |
| 25: P1     | 22: P3         |           |         |
| 32: P1, P3 | 27: { }        |           |         |
| 34: { }    |                |           |         |

Gantt Chart



I/O Gantt Chart



(ii)

| Process | Turnaround time | Waiting time       |
|---------|-----------------|--------------------|
| P1      | $30 - 4 = 26$   | $26 - 10 - 3 = 13$ |
| P2      | $28 - 0 = 28$   | $28 - 11 = 17$     |
| P3      | $34 - 6 = 28$   | $28 - 11 - 5 = 12$ |
|         | average = 27.33 | average = 14       |

c) You have designed an MLFQ with three total queues, the highest-priority queue being set with a time quantum of 50 ms, and the next two queues having time quanta of 200 ms and 500 ms, respectively. Now you are deciding on the promotion factors of the processes. You have two options in mind:

- If a process completes an I/O operation, it returns to the same level from which it came.
- It will get promoted to a higher priority level upon completing an I/O operation.

You have an interactive program that repeatedly performs the following from the very beginning after entering the queue: a short burst (30 ms) of CPU usage, and then waits for I/O to retrieve user information.

**Explain briefly with proper reasons whether there will be any differences in performance between the two choices for this process.**

### Given Situation

- MLFQ queues:
  - Q1 = 50 ms (highest)
  - Q2 = 200 ms
  - Q3 = 500 ms

- Process behavior: **30 ms CPU** → **I/O** → **repeat** (interactive, I/O-bound)

#### **Option (a): Return to same level after I/O**

- The process uses only **30 ms < 50 ms**, so it **never exhausts its quantum**.
- Hence, it **stays in the highest-priority queue (Q1)** permanently.
- Result: **Fast response time and good performance**.

#### **Option (b): Promote after I/O**

- After each I/O, the process is **promoted**, but it is already in **highest priority (Q1)**.
- So, promotion has **no practical effect**.
- Result: **Same fast response time and performance**.

**No difference in performance** for this process in both cases, because it always remains in the highest-priority queue due to its short CPU burst and frequent I/O.

#### ◆ **Remember**

Whenever you see:

- **CPU burst < smallest quantum**
- **Frequent I/O**

Instantly conclude:

- Process stays at **highest priority**
- **Promotion/demotion rules don't matter**

3. a) Write the answer in the question paper.

CA-M [2]

CO2 A programmer attempts to use a semaphore to protect a shared counter **x**, as shown below:

|                                                                                                  |                                                                                                  |
|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>semaphore mutex = 1;<br/>int x = 0;</code>                                                 |                                                                                                  |
| <code>// Thread 1<br/>int temp = x;<br/>wait(mutex);<br/>x = temp + 1;<br/>signal(mutex);</code> | <code>// Thread 2<br/>int temp = x;<br/>wait(mutex);<br/>x = temp + 1;<br/>signal(mutex);</code> |

Identify the synchronization mistake in this code.

Answer:

```
// Thread 1
wait(mutex); // Lock first
int temp = x; // Then read
x = temp + 1;
signal(mutex); // Then unlock
```

### The Core Issue

In thread synchronization, a **critical section** must encompass all operations on a shared resource- both reading and writing.

- **The Error:** The line `int temp = x;` occurs *before* `wait(mutex)`.
- **The Consequence:** Because the read happens before the lock is acquired, two threads can read the same initial value of **x** simultaneously. This leads to a **Race Condition** where one thread's update is overwritten (lost) by the other.

b) Explain whether it is always possible or not in a multicore system during multithreading to speed up a program by **n** times if we use **n** threads instead of one.

Answer:



No, it is **not always possible** to speed up a program by exactly  $n$  times using  $n$  threads on a multicore system. The speedup depends on several factors:

1. **Amdahl's Law:**

- If a fraction of the program is **serial** (cannot be parallelized), it limits the maximum speedup.
- Let  $P$  be the fraction of the program that can be parallelized, and  $(1-P)$  the serial part.
- Maximum speedup using  $n$  threads is:

$$\text{Speedup}_{\max} = \frac{1}{(1 - P) + \frac{P}{n}}$$

- Even with infinite threads, the serial portion prevents infinite speedup.

2. **Overheads:**

- Thread creation, context switching, synchronization (locks, semaphores) add extra overhead.
- Sometimes the overhead may **offset the benefits** of parallelism.

3. **Hardware limitations:**

- If the system has fewer than  $n$  cores, multiple threads share cores, reducing speedup.
- Memory bandwidth and cache contention can also slow down parallel threads.

**Conclusion:**

- In practice, **linear speedup** ( $n$  threads  $\rightarrow n \times$  faster) is **rare**.
- Speedup is usually **less than  $n$**  due to serial portions, overhead, and hardware limits.

c) Write the issues with proper logic that might arise if we use recursive functions in a multi-threaded program.

**Issues with Recursive Functions in Multi-Threaded Programs:**

1. **Stack Overflow:** Multiple threads with deep recursion can exhaust memory.

2. **Race Conditions:** Shared data accessed by recursive calls may get corrupted.
3. **Deadlocks / Synchronization Problems:** Nested recursion with locks can cause deadlocks.
4. **Performance Overhead:** Function call and thread management reduce efficiency.
5. **Hard to Debug:** Non-deterministic behavior makes errors difficult to trace.

d) Write the answer in the answer script.

The variable **counter** is a global variable that is shared by both **thread-1** and **thread-2**. The initial value of **counter** is 0, **temp1** and **temp2** are local variables. Consider **thread-1** and **thread-2** are run at the same time.

CA-D

| Code run by thread-1                                                                                     | Code run by thread-2                                                                                     |
|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <pre>// unrelated statements temp1 = counter; temp1 += 1; counter = temp1; // unrelated statements</pre> | <pre>// unrelated statements temp2 = counter; temp2 += 2; counter = temp2; // unrelated statements</pre> |

- I. **Identify** what kind of problem can arise in this scenario and how it can be avoided.
- II. **Modify** the codes of both **thread-1** and **thread-2** above by utilizing **two binary semaphores** to ensure that the final value of **counter** is exactly 2, after both threads have finished their execution. You have to mention the initial values of both semaphores.

[2]

[1.5+  
1.5=  
3]

i.

**Problem: Race Condition** The scenario describes a **Race Condition**. This occurs because both threads are accessing and modifying the shared global variable **counter** simultaneously without any synchronization mechanism.

Because the operations (read, increment, write) are not **atomic**, one thread can be preempted by the operating system after reading the value but before writing it

back. This leads to inconsistent results where one thread's update might "overwrite" or "lose" the update made by the other thread.

**How it can be avoided:** To avoid this, we must ensure **Mutual Exclusion** in the **Critical Section** (the lines of code where the shared variable is accessed). This can be achieved using:

- **Semaphores**
- **Mutex locks**

ii.

```
// unrelated statements
wait(S1);      // Wait until Thread-2 has read the initial counter value
```

```
temp1 = counter;
temp1 += 1;
counter = temp1; // Thread-1 finishes its update (counter becomes 1)
```

```
signal(S2);    // Tell Thread-2 it is now safe to overwrite the counter
// unrelated statements
```

```
// unrelated statements
temp2 = counter; // Thread-2 reads the initial 0 first
temp2 += 2;      // temp2 is now 2
```

```
signal(S1);    // Wake up Thread-1 and let it do its work
```

```
wait(S2);      // Wait here until Thread-1 has finished its update
counter = temp2; // Overwrite the counter with 2
// unrelated statements
```

S1=0

S2=0

*Solved by Azmari Sultana*